

```
// OpenCVApplication.cpp : Defines the entry point for the console application.
```

```
//
```

```
#include "stdafx.h"
```

```
#include "common.h"
```

```
#include <iostream>
```

```
#include <opencv2/opencv.hpp>
```

```
#include <vector>
```

```
#include <random>
```

```
#include <fstream>
```

```
using namespace cv;
```

```
using namespace std;
```

```
//-----Laborator 1-----//
```

```
void negativeImage()
```

```
{
```

```
    char fname[MAX_PATH];
```

```
    while (openFileDialog(fname))
```

```
    {
```

```
        Mat_<uchar> src;
```

```
        src = imread(fname, IMREAD_GRAYSCALE); //deschide imaginea ca imagine monocroma
```

```
        Mat_<uchar> dst = Mat_<uchar>(src.rows, src.cols);
```

```
        for (int i = 0; i < src.rows; i++) {
```

```
            for (int j = 0; j < src.cols; j++) {
```

```
                dst(i, j) = 255 - src(i, j);
```

```
            }
```

```
        }
```

```

        //pentru a vizualiza imaginea
        imshow("negative image", dst);

        waitKey();// se foloseste mereu dupa imshow
    }
}

//schimba nivelele de gri
void changeGreyLevelMul() {
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat_<uchar> src;
        src = imread(fname, IMREAD_GRAYSCALE);
        Mat_<uchar> dst = Mat_<uchar>(src.rows, src.cols);
        for (int i = 0; i < src.rows; i++) {
            for (int j = 0; j < src.cols; j++) {
                dst(i, j) = 2 * src(i, j);
            }
        }
        imshow("grey level multiplication", dst);
        imwrite("grelevelmultiplication.png", dst); //save the new image
        waitKey();
    }
}

//crearea unei imagini de dimensiune 256x256. Impartirea imaginii
//in 4 cadrane egale si colorarea lor din stanga-sus pana in
//dreapta jos astfel: alb, rosu, verde, galben

```

```

void createImage() {
    Mat img(256, 256, CV_8UC3);
    for (int i = 0; i < 128; i++) {
        for (int j = 0; j < 128; j++) {
            //pt a accesa pixelul de la coordonatele (i,j)
            img.at<Vec3b>(i, j) = Vec3b(255, 0, 0);
        }
        for (int j = 128; j < 255; j++) {
            img.at<Vec3b>(i, j) = Vec3b(0, 255, 0);
        }
    }
    for (int i = 128; i < 255; i++) {
        for (int j = 0; j < 128; j++) {
            img.at<Vec3b>(i, j) = Vec3b(0, 0, 255);
        }
        for (int j = 128; j < 255; j++) {
            img.at<Vec3b>(i, j) = Vec3b(0, 0, 0);
        }
    }
    imshow("input image", img);
    waitKey();
}

```

//creare matrice 3x3 de tip float. Determinarea inversei si tiparirea ei

```

void create3x3InversMatrix() {
    Mat_<float> matrix(3, 3, CV_32FC1);
    matrix(0, 0) = 0;
    matrix(0, 1) = 0;
    matrix(0, 2) = 0;

```

```

matrix(1, 0) = 1;
matrix(1, 1) = 1;
matrix(1, 2) = 1;
matrix(2, 0) = 1;
matrix(2, 1) = 1;
matrix(2, 2) = 1;
for (int i = 0; i < matrix.rows; i++) {
    for (int j = 0; j < matrix.cols; j++) {
        std::cout << matrix(i, j) << " ";
    }
    std::cout << std::endl;
}
std::cout << std::endl;
std::cout << std::endl;
Mat_<float> inverse = matrix.inv();
for (int i = 0; i < inverse.rows; i++) {
    for (int j = 0; j < inverse.cols; j++) {
        std::cout << inverse(i, j) << " ";
    }
    std::cout << std::endl;
}
waitKey(4000);
}

```

//-----Laborator 2-----//

//copiază canalele R,G,B ale unei imagini RGB24(TIP cv_8uc3)-

```

//aici fiecare culoare e reprezentata pe cate 8 biti(256 de nivele)
//pt imagini RGB24(24 biti/pixel) spatiul de culoare poate fi reprezentat complet
//in 3 matrice de tip CV_8UC1(8 biti/pixel)
//afisati aceste matrice in 3 ferestre diferite
void fromUC3to3UC1() {
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat_<Vec3b> src;
        src = imread(fname, IMREAD_COLOR); //open the image
        Mat_<uchar> dst_r = Mat_<uchar>(src.rows, src.cols);
        Mat_<uchar> dst_b = Mat_<uchar>(src.rows, src.cols);
        Mat_<uchar> dst_g = Mat_<uchar>(src.rows, src.cols);
        for (int i = 0; i < src.rows; i++) {
            for (int j = 0; j < src.cols; j++) {
                dst_b(i, j) = src(i, j)[0]; //blue
                dst_g(i, j) = src(i, j)[1]; //green
                dst_r(i, j) = src(i, j)[2]; //red
            }
        }
        imshow("blue image", dst_b);
        imshow("red image", dst_r);
        imshow("green image", dst_g);
        waitKey();
    }
}

```

```

//conversie de la o imagine color RGB24(tip CV_8UC3) la o imagine grayscale

```

```
//de tip CV_8UC1
```

```
//cele trei componente ale culorii fiecarui pixel trebuie egalizate
```

```
void fromRGBtoGrayscale() {  
    char fname[MAX_PATH];  
    while (openFileDialog(fname))  
    {  
        Mat_<Vec3b> src;  
        src = imread(fname, IMREAD_COLOR); //open the image  
        Mat_<uchar> dst = Mat_<uchar>(src.rows, src.cols);  
        float media;  
        for (int i = 0; i < src.rows; i++) {  
            for (int j = 0; j < src.cols; j++) {  
                //compute the intensity as the average of the three channels  
                media = src(i, j)[0] + src(i, j)[1] + src(i, j)[2];  
                dst(i, j) = media;  
            }  
        }  
  
        imshow("grayscale image", dst);  
        waitKey();  
    }  
}
```

```
//conversie din grayscale in alb-negru
```

```
void fromGrayscaleToBW(int threshold) {  
    /*char fname[MAX_PATH];  
    while (openFileDialog(fname))  
    {
```

```

Mat_<uchar> src;

src = imread(fname, IMREAD_GRAYSCALE); //open the image

Mat_<uchar> dst = Mat_<uchar>(src.rows, src.cols);

for (int i = 0; i < src.rows; i++) {
    for (int j = 0; j < src.cols; j++) {
        if (src(i, j) > threshold) dst(i, j) = 255;
        else dst(i, j) = 0;
    }
}

imshow("binary image", dst);

waitKey();

}*/

char fname[MAX_PATH];

while (openFileDialog(fname)) {
    Mat grey = imread(fname, IMREAD_GRAYSCALE);
    int height = grey.rows;
    int width = grey.cols;

    int prag;
    std::cout << "Valoare prag: ";
    std::cin >> prag;

    Mat black = Mat(height, width, CV_8UC1);

    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (grey.at<uchar>(i, j) < prag) {
                black.at<uchar>(i, j) = 0;
            }
        }
    }
}

```

```

        }
        else {
            black.at<uchar>(i, j) = 255;
        }
    }
}

imshow("greyscale", grey);
imshow("black and white", black);
waitKey();
}
}

```

```

float max3(float a, float b, float c)
{
    float max = 0;
    if (a > b) max = a;
    else max = b;
    if (c > max) max = c;
    return max;
}

```

```

float min3(float a, float b, float c)
{
    float min = 255;
    if (a > b) min = b;
    else min = a;
    if (c < min) min = c;
}

```

```

        return min;

    }

//converteste canalele R,G,B ale unei imagini RGB24 in componente H,S,V
void computeHSVfromRGB() {

    float b, g, r;
    float B, G, R;
    float M, m, C;
    float H = 0, S, V;
    float H_norm, S_norm, V_norm;

    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat_<Vec3b> src;
        src = imread(fname, IMREAD_COLOR); //open the image
        Mat_<uchar> dst_H = Mat_<uchar>(src.rows, src.cols);
        Mat_<uchar> dst_S = Mat_<uchar>(src.rows, src.cols);
        Mat_<uchar> dst_V = Mat_<uchar>(src.rows, src.cols);
        for (int i = 0; i < src.rows; i++) {
            for (int j = 0; j < src.cols; j++) {

                B = src(i, j)[0];
                G = src(i, j)[1];
                R = src(i, j)[2];

                r = R / 255; // r : the normalized R component
            }
        }
    }
}

```

```
g = G / 255; // g : the normalized G component
```

```
b = B / 255; // b : the normalized B component
```

```
M = max3(r, g, b);
```

```
m = min3(r, g, b);
```

```
C = M - m;
```

```
//value
```

```
V = M;
```

```
//saturation
```

```
if (V != 0)
```

```
    S = C / V;
```

```
else // black
```

```
    S = 0;
```

```
//hue
```

```
if (C != 0) {
```

```
    if (M == r) H = 60 * (g - b) / C;
```

```
    if (M == g) H = 120 + 60 * (b - r) / C;
```

```
    if (M == b) H = 240 + 60 * (r - g) / C;
```

```
}
```

```
else // grayscale
```

```
    H = 0;
```

```
if (H < 0)
```

```
    H = H + 360;
```

```
//scale
```

```
H_norm = H * 255 / 360;
```

```

        S_norm = S * 255;
        V_norm = V * 255;

        dst_H(i, j) = H_norm;
        dst_V(i, j) = V_norm;
        dst_S(i, j) = S_norm;

    }

}

imshow("H", dst_H);
imshow("S", dst_S);
imshow("V", dst_V);
waitKey();
}
}

bool isPointInside(Mat img, int i, int j) {

    if (i > img.rows || j > img.cols || i < 0 || j < 0) return false;
    else return true;

}

//verifica daca pozitia indicata de perechea (i,j)(rand,coloana) e inuntrul imaginii de mai jos
void testIsPointInside(int i, int j) {

    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat_<Vec3b> src;

```

```

        src = imread(fname, IMREAD_COLOR); //open the image

        if (isPointInside(src, i, j)) printf("The point is inside");

        else printf("The point is NOT inside");

        waitKey();

    }

}

//-----Laborator 3-----

//histograma-algoritmul din laborator

void showHistogram(const std::string& name, int* hist, const int hist_cols, const int hist_height)
{
    Mat imgHist(hist_height, hist_cols, CV_8UC3, CV_RGB(255, 255, 255)); // constructs a white
image

    //computes histogram maximum

    int max_hist = 0;

    for (int i = 0; i < hist_cols; i++)
        if (hist[i] > max_hist)
            max_hist = hist[i];

    double scale = 1.0;

    scale = (double)hist_height / max_hist;

    int baseline = hist_height - 1;

    for (int x = 0; x < hist_cols; x++) {
        Point p1 = Point(x, baseline);

        Point p2 = Point(x, baseline - cvRound(hist[x] * scale));

        line(imgHist, p1, p2, CV_RGB(255, 0, 255)); // histogram bins colored in magenta
    }
}

```

```

        imshow(name, imgHist);
    }

//calculare FDP si afisarea ei
void computeHistogramFDPandShow(){
    int histogram[256] = { 0 };
    float pdf[256];
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat_<uchar> src;
        src = imread(fname, IMREAD_GRAYSCALE); //open the image

        //compute histogram
        for (int i = 0; i < src.rows; i++) {
            for (int j = 0; j < src.cols; j++) {
                histogram[src(i, j)]++;
            }
        }

        //compute pdf
        float M = src.rows * src.cols;
        for (int i = 0; i < 256; i++) {
            pdf[i] = (float)histogram[i] / M;
        }

        showHistogram("Histogram", histogram, 256, 256);
        waitKey();
    }
}

```

```
    }  
}
```

//praguri multiple

```
void multiLevelThreshold() {  
    char fname[MAX_PATH];  
    int histogram[256] = { 0 };  
    float pdf[256];  
    int maxHistogram[256];  
    while (openFileDialog(fname))  
    {  
        Mat_<uchar> src;  
        src = imread(fname, IMREAD_GRAYSCALE); //open the image  
        Mat_<uchar> dst = Mat_<uchar>(src.rows, src.cols);  
  
        //compute histogram  
        for (int i = 0; i < src.rows; i++) {  
            for (int j = 0; j < src.cols; j++) {  
                histogram[src(i, j)]++;  
            }  
        }  
  
        //compute pdf  
        float M = src.rows * src.cols;  
        for (int i = 0; i < 256; i++) {  
            pdf[i] = (float)histogram[i] / M;  
        }  
  
        //compute MaxHistogram
```

```

int wh = 5;

float th = 0.0003;

maxHistogram[0] = 0;

maxHistogram[255] = 256;

int index = 1;

for (int k = wh; k <= 255 - wh; k++)
{
    float v = 0;

    for (int i = k - wh; i <= k + wh; i++)
    {
        v += pdf[i];
    }

    float average = v / (2 * wh + 1);

    if (pdf[k] > average + th)
    {
        bool largest = true;

        for (int i = k - wh; i <= k + wh; i++)
        {
            if (pdf[k] < pdf[i])
            {
                largest = false;
                break;
            }
        }

        if (largest)
        {
            maxHistogram[index] = k;
            index++;
        }
    }
}

```

```

        }
    }
}

for (int i = 0; i < src.rows; i++) {
    for (int j = 0; j < src.cols; j++) {
        int minValue = 255;
        int minDif = 255;

        for (int l = 0; l < 256; l++) {
            if (abs(src(i, j) - maxHistogram[l]) < minDif) {
                minDif = abs(src(i, j) - maxHistogram[l]);
                minValue = maxHistogram[l];
            }
        }
        dst(i, j) = minValue;
    }
}

imshow("Input", src);
imshow("Output-Multi Level", dst);
waitKey();
}
}

```

//////////-----Laborator 4-----//////////

```

void afisareCulori(int x, int y, Mat* img) {

    int a = (int)(*img).at<Vec3b>(y, x)[2];
    int b = (int)(*img).at<Vec3b>(y, x)[1];
    int c = (int)(*img).at<Vec3b>(y, x)[0];
    printf("%d %d %d\n", a, b, c);
    if (a == 128 && b == 128 && c == 128) {
        printf("Ai selectat figura gri!\n");
    }
    if (a == 255 && b == 0 && c == 0) {
        printf("Ai selectat figura rosie!\n");
    }
    if (a == 0 && b == 0 && c == 255) {
        printf("Ai selectat figura albastru intens!\n");
    }
    if (a == 0 && b == 128 && c == 192) {
        printf("Ai selectat figura albastru sters!\n");
    }
    if (a == 0 && b == 255 && c == 0) {
        printf("Ai selectat figura verde!\n");
    }
    if (a == 128 && b == 64 && c == 0) {
        printf("Ai selectat figura maro!\n");
    }
    if (a == 255 && b == 0 && c == 255) {
        printf("Ai selectat figura mov!\n");
    }
}

```

```

int arieFigura(int x, int y, Mat* img) {

```

```

int arie = 0;
for (int i = 0; i < img->rows; i++) {
    for (int j = 0; j < img->cols; j++) {
        Vec3b culoareSelectata = (Vec3b)(*img).at<Vec3b>(y, x);
        if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {
            arie += 1;
        }
    }
}
return arie;
}

```

```

void centruDeMasa(int x, int y, int arie, Mat* img, float* centru, Mat dst) {
    float r = 0, c = 0;
    for (int i = 0; i < img->rows; i++) {
        for (int j = 0; j < img->cols; j++) {
            Vec3b culoareSelectata = (Vec3b)(*img).at<Vec3b>(y, x);
            if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {
                r += i;
                c += j;
            }
        }
    }
    r *= (1.0 / arie);
    c *= (1.0 / arie);
    centru[0] = r;
    centru[1] = c;
    dst.at<Vec3b>(r, c) = Vec3b(0, 0, 0);
}

```

```

float axaAlungire(int x, int y, float* centru, Mat* img, Mat dst) {

    int numarator = 0, numitor = 0;

    Vec3b culoareSelectata = (Vec3b)(*img).at<Vec3b>(y, x);

    Point point1, point2;

    for (int i = 0; i < img->rows; i++) {
        for (int j = 0; j < img->cols; j++) {
            if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {
                numarator += (i - (int)centru[0]) * (j - (int)centru[1]);
                numitor += (pow((j - (int)centru[1]), 2)) - (pow((i - (int)centru[0]), 2));
            }
        }
    }

    float unghi = (atan2(2 * numarator, numitor) * 180 / PI) / 2;

    if (unghi < 0) {
        unghi += 180;
    }

    point1 = Point(centru[1] - 60 * cos(unghi * PI / 180.0), centru[0] - 60 * sin(unghi * PI / 180));
    point2 = Point(centru[1] + 60 * cos(unghi * PI / 180), centru[0] + 60 * sin(unghi * PI / 180));
    line(dst, point1, point2, Scalar(255, 192, 203));

    return unghi;
}

```

```

float elongatie(int x, int y, Mat* img) {
    int row_maxim = 0, row_minim = 999999999, col_maxim = 0, col_minim = 999999999;
    for (int i = 0; i < img->rows; i++) {
        for (int j = 0; j < img->cols; j++) {
            Vec3b culoareSelectata = (Vec3b)(*img).at<Vec3b>(y, x);
            if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {
                if (i > row_maxim) {
                    row_maxim = i;
                }
                if (i < row_minim) {
                    row_minim = i;
                }
                if (j > col_maxim) {
                    col_maxim = j;
                }
                if (j < col_minim) {
                    col_minim = j;
                }
            }
        }
    }

    float R = (float)(col_maxim - col_minim + 1) / (row_maxim - row_minim + 1);
    return R;
}

```

```

int perimetruAprox(int x, int y, Mat* img, Mat dst) {
    int perimetru = 0;
    int dx[9] = { -1,-1,0,1,1,1,0,-1,0 };
    int dy[9] = { 0,1, 1,1,0,-1,-1,-1,0 };

```

```

for (int i = 1; i < img->rows - 1; i++) {
    for (int j = 1; j < img->cols - 1; j++) {
        Vec3b culoareSelectata = (Vec3b)(*img).at<Vec3b>(y, x);
        if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {
            for (int k = 0; k < 8; k++) {
                if ((Vec3b)(*img).at<Vec3b>(i + dx[k], j + dy[k]) == Vec3b(255,
255, 255)) {

                    perimetru++;
                    dst.at<Vec3b>(i, j) = Vec3b(0, 0, 0);
                    break;
                }
            }
        }
    }
}

return perimetru;
}

```

```

void creareProiectii(Mat* img, Mat proiectii, int x, int y) {

```

```

    int h = img->rows;
    int w = img->cols;
    int* pLinie = (int*)malloc(h * sizeof(int));
    int* pColoane = (int*)malloc(w * sizeof(int));

    for (int i = 0; i < h; i++) {
        pLinie[i] = 0;
    }

```

```
for (int i = 0; i < w; i++) {  
    pColoane[i] = 0;  
}
```

```
for (int i = 0; i < img->rows; i++) {  
    int pixeliLinie = 0;  
    for (int j = 0; j < img->cols; j++) {  
        if ((Vec3b)(*img).at<Vec3b>(y, x) == (Vec3b)(*img).at<Vec3b>(i, j)) {  
            pixeliLinie++;  
            pColoane[j]++;  
        }  
    }  
    pLinie[i] = pixeliLinie;  
}
```

```
for (int i = 0; i < h; i++) {  
    if (pLinie[i] != 0) {  
        for (int j = 0; j < pLinie[i]; j++) {  
            proiectii.at<Vec3b>(i, j) = (Vec3b)(*img).at<Vec3b>(y, x);  
        }  
    }  
}
```

```
for (int i = 0; i < w; i++) {  
    if (pColoane[i] > 0) {  
        for (int j = 0; j < pColoane[i]; j++) {  
            proiectii.at<Vec3b>(h - j - 1, i) = (Vec3b)(*img).at<Vec3b>(y, x);  
        }  
    }  
}
```

```

    }

    imshow("Proiectia horizontala si cea verticala", proiectii);
    waitKey(0);
}

void onMouseGeometricalFeatures(int event, int x, int y, int flags, void* param) {
    Mat* img = (Mat*)param;

    Mat dst = Mat(img->rows, img->cols, CV_8UC3);
    Mat proiectii = Mat(img->rows, img->cols, CV_8UC3);

    for (int i = 0; i < img->rows; i++) {
        for (int j = 0; j < img->cols; j++) {
            dst.at<Vec3b>(i, j) = Vec3b(255, 255, 255);
        }
    }

    if (event == EVENT_LBUTTONDOWN)
    {
        afisareCulori(x, y, img);
        int arie = arieFigura(x, y, img);
        printf("Aria figurii este : %d\n", arie);
        float* centru = (float*)malloc(sizeof(float) * 2);
        centruDeMasa(x, y, arie, img, centru, dst);
        printf("Centrul de masa are coordonatele: (%d, %d)\n", (int)centru[0], (int)centru[1]);
        float unghiAlungire = axaAlungire(x, y, centru, img, dst);
        printf("Unghi alungire : %d\n", (int)unghiAlungire);
    }
}

```

```

        float R = elongatie(x, y, img);
        printf("Elongatia: %f\n", R);
        int perimetru = perimetruAprox(x, y, img, dst) * PI / 4;
        printf("Perimetru: %d\n", perimetru);
        printf("Factorul de subtiere: %f", 4 * PI * ((float)arie / (perimetru * perimetru)));
        creareProiectii(img, proiectii, x, y);
        imshow("Puncte contur, Centru de masa, axa de alungire", dst);
        waitKey(0);
    }

}

```

```

void testComputeGeometricalFeatures() {
    char fname[MAX_PATH];
    while (openFileDialog(fname)) {
        Mat img = imread(fname, IMREAD_COLOR);
        namedWindow("My Window", 1);
        setMouseCallback("My Window", onMouseGeometricalFeatures, &img);
        imshow("My Window", img);
        waitKey(0);
    }
}

```

//////////-----Laborator 5-----////////

//traversarea in latime

```

/*void bfs() {
    char fname[MAX_PATH];

```

```

while (openFileDialog(fname)) {
    Mat_<uchar> image = imread(fname, IMREAD_GRAYSCALE);
    imshow("Display Image", image);
    int label = 0;
    Mat_<int> labels(image.rows, image.cols);
    labels = 0;
    vector<vector<int>> edges;
    edges.push_back(vector<int>());

    int di[] = { 0, -1, -1, -1 };
    int dj[] = { -1, -1, 0, 1 };

    for (int i = 0; i < image.rows; i++) // RADNDURI
        for (int j = 0; j < image.cols; j++) { // COLOANE
            if ((image(i, j) == 0) && (labels(i, j) == 0)) {
                //          printf("gasit %d %d\n", i, j);
                vector<int> L;
                for (int k = 0; k < 4; k++) {
                    int iv = i + di[k];
                    int jv = j + dj[k];
                    if ((0 <= iv) && (iv < image.rows) && (0 <= jv) && (jv <=
image.cols)) {

                        if (labels(iv, jv) > 0) {
                            L.push_back(labels(iv, jv));
                        }
                    }//if neighbor in the image
                }//for all neighbors
                if (L.size() == 0) {
                    label++;
                }
            }
        }
    }
}

```

```

        edges.push_back(vector<int>());
        labels(i, j) = label;
    }
    else {
        int x = L[0];
        for (int k = 1; k < L.size(); k++) {
            x = min(x, L[k]);
        }
        labels(i, j) = x;
        for (int k = 1; k < L.size(); k++) {
            int y = L[k];
            if (y != x) {
                edges[x].push_back(y);
                edges[y].push_back(x);
            }
        }
    }
}

} //if pixel object
}

```

```

int newlabel = 0;
vector<int> newlabels(label + 1, 0);
for (int i = 1; i < label; i++) {
    if (newlabels[i] == 0) {
        newlabel++;
        queue<int> Q;
        newlabels[i] = newlabel;
        Q.push(i);
    }
}

```

```

        while (Q.size() > 0) {
            int x = Q.front();
            Q.pop();
            for (int k = 0; k < edges[x].size(); k++) {
                int y = edges[x][k];
                if (newlabels[y] == 0) {
                    newlabels[y] = newlabel;
                    Q.push(y);
                }
            }
        }
    }
}

```

```

default_random_engine gen;
uniform_int_distribution<int> d(0, 255);

```

```

#define MAX_CULORI 400

```

```

vector<Vec3b> index_culori(MAX_CULORI);
index_culori[0] = Vec3b(255, 255, 255);
for (int k = 1; k < MAX_CULORI; k++) {
    index_culori[k][0] = d(gen);
    index_culori[k][1] = d(gen);
    index_culori[k][2] = d(gen);
}

```

```

Mat_<Vec3b> etichete_colorate(image.rows, image.cols);
for (int i = 0; i < image.rows; i++)

```

```

        for (int j = 0; j < image.cols; j++) {
            etichete_colorate(i, j) = index_culori[newlabels[labels(i, j)] %
MAX_CULORI];
        }

        imshow("etichete", etichete_colorate);

        waitKey(0);
    }
}*/

```

```

void traversareInaltime(Mat img, int* di, int* dj, int n, Mat dst) {
    int label = 0;

    int** labels = (int**)calloc(img.rows, sizeof(int*));

    for (int i = 0; i < img.rows; i++) {
        labels[i] = (int*)calloc(img.cols, sizeof(int));
    }

    default_random_engine gen;

    uchar random1, random2, random3;

    uniform_int_distribution<int> d(0, 255);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            if (img.at<uchar>(i, j) == 0 && labels[i][j] == 0) {

                label++;

                random1 = d(gen);
                random2 = d(gen);
                random3 = d(gen);

                queue <Point2i> Q;

                labels[i][j] = label;

                Q.push({ j,i });
            }
        }
    }
}

```

```

        while (!Q.empty()) {
            Point2i q = Q.front();
            Q.pop();
            int neighborI, neighborJ;
            for (int k = 0; k < n; k++) {
                neighborI = q.y + dj[k];
                neighborJ = q.x + di[k];
                if (neighborI < img.rows && neighborI >= 0 &&
neighborbotJ < img.cols && neighborJ >= 0) {
                    if (img.at<uchar>(neighborI, neighborJ) == 0 &&
labels[neighborI][neighborJ] == 0) {
                        Vec3b(random1, random2, random3);
                        dst.at<Vec3b>(neighborI, neighborJ) =
labels[neighborI][neighborJ] = label;
                        Q.push({ neighborJ,neighborI });
                    }
                }
            }
        }
    }
}

```

```

void generateColorFromBFS(Mat img)
{

```

```

    int height = img.rows;

```

```

    int width = img.cols;

```

```

Mat dst1 = Mat(height, width, CV_8UC3);
Mat dst2 = Mat(height, width, CV_8UC3);

int dj[8] = { -1, -1, -1, 0, 1, 1, 1, 0 };
int di[8] = { -1, 0, 1, 1, 1, 0, -1, -1 };

traversareInaltime(img, di, dj, 8, dst1);
traversareInaltime(img, dj, di, 4, dst2);

imshow("Color (N8) From BFS", dst1);
imshow("Color (N4) From BFS", dst2);
waitKey(0);
}

```

```

void douaTreceri() {
    char fname[MAX_PATH];
    openFileDialog(fname);

    default_random_engine gen;
    uniform_int_distribution<int> d(0, 255);
    uchar c1, c2, c3;

    Mat img = imread(fname, IMREAD_GRAYSCALE);

    int height = img.rows;
    int width = img.cols;

```

```
int label = 0;
```

```
int dj[8] = { -1,-1,-1,0 };
```

```
int di[8] = { -1,0,1,-1 };
```

```
int** labels = (int**)calloc(height, sizeof(int*));
```

```
for (int i = 0; i < height; i++) {
```

```
    labels[i] = (int*)calloc(width, sizeof(int));
```

```
}
```

```
Mat dst = Mat(height, width, CV_8UC3);
```

```
vector<vector<int>> edges; //vecinii pentru fiecare punct
```

```
// prima trecere
```

```
for (int i = 0; i < height; i++) {
```

```
    for (int j = 0; j < width; j++) {
```

```
        if (img.at<uchar>(i, j) == 0 && labels[i][j] == 0) {
```

```
            vector<int> L; //daca nu ar fi aici ar trebui sa facem pop
```

```
            for (int k = 0; k < 8; k++) {
```

```
                int ni = i + dj[k];
```

```
                int nj = j + di[k];
```

```
                if (ni < height && ni >= 0 && nj < width && nj >= 0) {
```

```
                    if (labels[ni][nj] > 0) {
```

```
                        L.push_back(labels[ni][nj]); //vecini etichetati
```

```
                    }
```

```
            }
```

eticheta

```
    }

    if (L.empty()) { //daca nu sunt vecini etichetati se creeaza o noua
        label++;
        edges.resize(label + 1);
        labels[i][j] = label;
    }

    else { // daca sunt vecini etichetati se selecteaza minimul etichetelor lor
        si fiecare eticheta diferita de minim, va fi echivalenta cu minimul

        int min = INT_MAX;

        for (int elem : L) { //se cauta minimul etichetelor
            if (elem < min) {
                min = elem;
            }
        }

        int x = min;
        edges.resize(label + 1);
        labels[i][j] = x;

        for (int elem : L) {
            if (x != elem) {
                edges[x].push_back(elem);
                edges[elem].push_back(x);
            }
        }
    }
}
```

```

    }
}

int* newLabels = (int*)calloc(label + 1, sizeof(int));
int newLabel = 0;

for (int i = 1; i < label; i++)
{
    if (newLabels[i] == 0)
    {
        newLabel++;
        queue<int> Q;
        newLabels[i] = newLabel;
        Q.push(i);

        while (!Q.empty())
        {
            int q = Q.front();
            Q.pop();

            for (int k : edges[q]) {
                if (newLabels[k] == 0)
                {
                    newLabels[k] = newLabel;
                    Q.push(k);
                }
            }
        }
    }
}

```

```
}
```

```
for (int i = 0; i < height; i++) {  
    for (int j = 0; j < width; j++) {  
        labels[i][j] = newLabels[labels[i][j]];  
    }  
}
```

```
vector<Vec3b> newRandomColors(label + 1, 0);
```

```
newRandomColors.at(0) = Vec3b(255, 255, 255);
```

```
for (int i = 1; i <= label; i++) {  
    c1 = d(gen);  
    c2 = d(gen);  
    c3 = d(gen);  
  
    newRandomColors.at(i) = Vec3b(c1, c2, c3);  
}
```

```
for (int i = 0; i < height; i++) {  
    for (int j = 0; j < width; j++) {  
        dst.at<Vec3b>(i, j) = newRandomColors.at(labels[i][j]);  
    }  
}
```

```
imshow("Doua treceri", dst);
```

```
generateColorFromBFS(img);
```

```
waitKey();
```

```
}
```

```
//-----Laborator 6-----//
```

```
//algoritmul de urmarire a conturului si desenarea conturului obiectului
```

```
//dintr-o imagine avand un singur obiect
```

```
void borderTracing()
```

```
{
```

```
    char fname[MAX_PATH];
```

```
    while (openFileDialog(fname)) {
```

```
        Mat_<uchar> img = imread(fname, IMREAD_GRAYSCALE);
```

```
        vector<Point2i> contour(img.rows * img.cols);
```

```
        Point2i p0, p1;
```

```
        int dir = 7;
```

```
        vector<int> ac(img.rows * img.cols);
```

```
        vector<int> dc(img.rows * img.cols);
```

```
        int n = 0;
```

```
        int dy[8] = { -1, -1, -1, 0, 1, 1, 1, 0 };
```

```
        int dx[8] = { -1, 0, 1, 1, 1, 0, -1, -1 };
```

```
        int ni = 0, nj = 0;
```

```
        for (int i = 0; i < img.rows; i++)
```

```
        {
```

```
            for (int j = 0; j < img.cols; j++)
```

```
            {
```

```
                if (img.at<uchar>(i, j) == 0)
```

```

        {

            p0 = Point2i(i, j);

            ni = i;

            nj = j;


            cout << i << " " << j << endl;


            i = img.rows;
            j = img.cols;

        }

    }

}

contour.at(0) = p0;
ac.at(0) = dir;

do {

    if (dir % 2 == 0)

    {

        dir = (dir + 7) % 8;

    }

    else

    {

        dir = (dir + 6) % 8;

    }


    while (img.at<uchar>(ni + dy[dir], nj + dx[dir]) != 0)

    {

        dir = (dir + 1) % 8;

```

```

    }

    n++;
    ac.at(n) = dir;
    dc.at(n - 1) = (ac.at(n) - ac.at(n - 1) + 8) % 8;

    ni += dy[dir];
    nj += dx[dir];

    Point2i pN = Point2i(ni, nj);

    if (n == 1)
    {
        p1 = Point2i(ni, nj);
        contour.at(1) = p1;
    }
    else {
        contour.at(n) = pN;
    }
} while (n < 2 || contour.at(n) != p1 || contour.at(n - 1) != p0);

Mat outputImg(img.rows, img.cols, CV_8UC3, Scalar(255, 255, 255));

cout << "AC: ";
for (int k = 0; k < n; k++)
{
    cout << ac.at(k) << " ";
}
cout << endl;

```

```

        cout << "DC: ";
        for (int k = 0; k < n; k++)
        {
            cout << dc.at(k) << " ";
        }
        cout << endl;

        for (int i = 0; i < img.rows; i++)
        {
            for (int j = 0; j < img.cols; j++)
            {
                for (int k = 0; k < n; k++)
                {
                    if (contour.at(k) == Point2i(i, j))
                    {
                        outputImg.at<Vec3b>(i, j) = Vec3b(0, 0, 255);
                    }
                }
            }
        }

        imshow("input", img);
        imshow("output", outputImg);
        waitKey(0);
    }
}

```

//algoritmul de urmarire a conturului

//construieste codul inlantuit si derivata codului inlantuit al unui obiect

```

void contourReconstruction()
{
    ifstream file;
    file.open("/Users/Armina/Desktop/reconstruct.txt");
    Mat img = imread("/Users/Armina/gray_background.bmp", IMREAD_GRAYSCALE);

    if (!file)
    {
        cout << "Unable to open the file" << endl;
    }
    else
    {
        int size = 0;
        Point2i startingPoint = Point2i(0, 0);

        file >> startingPoint.x;
        file >> startingPoint.y;
        file >> size;

        int dy[8] = { 0, -1, -1, -1, 0, 1, 1, 1 };
        int dx[8] = { 1, 1, 0, -1, -1, -1, 0, 1 };

        for (int i = 0; i < size; i++) {

            int dir;
            file >> dir;

            img.at<uchar>(startingPoint.x, startingPoint.y) = 0;

```

```

        startingPoint.x += dy[dir];
        startingPoint.y += dx[dir];
    }

    imshow("output", img);
    waitKey(0);
}
}

```

```

void testOpenImage()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat src;
        src = imread(fname);
        imshow("image", src);
        waitKey();
    }
}

```

```

void testOpenImagesFld()
{
    char folderName[MAX_PATH];
    if (openFolderDlg(folderName) == 0)
        return;

    char fname[MAX_PATH];
    FileGetter fg(folderName, "bmp");
}

```

```

while (fg.getNextAbsFile(fname))
{
    Mat src;

    src = imread(fname);

    imshow(fg.getFoundFileName(), src);

    if (waitKey() == 27) //ESC pressed
        break;
}

}

void testImageOpenAndSave()
{
    Mat src, dst;

    src = imread("Images/Lena_24bits.bmp", IMREAD_COLOR);    // Read the image

    if (!src.data)    // Check for invalid input
    {
        printf("Could not open or find the image\n");
        return;
    }

    // Get the image resolution
    Size src_size = Size(src.cols, src.rows);

    // Display window
    const char* WIN_SRC = "Src"; //window for the source image
    namedWindow(WIN_SRC, WINDOW_AUTOSIZE);
    moveWindow(WIN_SRC, 0, 0);

```

```

const char* WIN_DST = "Dst"; //window for the destination (processed) image
namedWindow(WIN_DST, WINDOW_AUTOSIZE);
moveWindow(WIN_DST, src_size.width + 10, 0);

cvtColor(src, dst, COLOR_BGR2GRAY); //converts the source image to a grayscale one

imwrite("Images/Lena_24bits_gray.bmp", dst); //writes the destination to file

imshow(WIN_SRC, src);
imshow(WIN_DST, dst);

printf("Press any key to continue ...\n");
waitKey(0);
}

void testNegativeImage()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        double t = (double)getTickCount(); // Get the current time [s]

        Mat src = imread(fname, IMREAD_GRAYSCALE);
        int height = src.rows;
        int width = src.cols;
        Mat dst = Mat(height, width, CV_8UC1);

        // Asa se acceseaza pixelii individuali pt. o imagine cu 8 biti/pixel
        // Varianta ineficienta (lenta)

```

```

        for (int i = 0; i < height; i++)
        {
            for (int j = 0; j < width; j++)
            {
                uchar val = src.at<uchar>(i, j);
                uchar neg = 255 - val;
                dst.at<uchar>(i, j) = neg;
            }
        }

        // Get the current time again and compute the time difference [s]
        t = ((double)getTickCount() - t) / getTickFrequency();
        // Print (in the console window) the processing time in [ms]
        printf("Time = %.3f [ms]\n", t * 1000);

        imshow("input image", src);
        imshow("negative image", dst);
        waitKey();
    }
}

void testParcureSimplaDiblookStyle()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat src = imread(fname, IMREAD_GRAYSCALE);
        int height = src.rows;
        int width = src.cols;
    }
}

```

```

Mat dst = src.clone();

double t = (double)getTickCount(); // Get the current time [s]

// the fastest approach using the "diblook style"
uchar* lpSrc = src.data;
uchar* lpDst = dst.data;
int w = (int)src.step; // no dword alignment is done !!!
for (int i = 0; i < height; i++)
    for (int j = 0; j < width; j++) {
        uchar val = lpSrc[i * w + j];
        lpDst[i * w + j] = 255 - val;
    }

// Get the current time again and compute the time difference [s]
t = ((double)getTickCount() - t) / getTickFrequency();
// Print (in the console window) the processing time in [ms]
printf("Time = %.3f [ms]\n", t * 1000);

imshow("input image", src);
imshow("negative image", dst);
waitKey();
}

}

void testColor2Gray()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))

```

```

{
    Mat src = imread(fname);

    int height = src.rows;
    int width = src.cols;

    Mat dst = Mat(height, width, CV_8UC1);

    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            Vec3b v3 = src.at<Vec3b>(i, j);
            uchar b = v3[0];
            uchar g = v3[1];
            uchar r = v3[2];
            dst.at<uchar>(i, j) = (r + g + b) / 3;
        }
    }

    imshow("input image", src);
    imshow("gray image", dst);
    waitKey();
}

void testBGR2HSV()
{

```

```

char fname[MAX_PATH];
while (openFileDialog(fname))
{
    Mat src = imread(fname);

    int height = src.rows;
    int width = src.cols;


    Mat H = Mat(height, width, CV_8UC1);
    Mat S = Mat(height, width, CV_8UC1);
    Mat V = Mat(height, width, CV_8UC1);


    uchar* lpH = H.data;
    uchar* lpS = S.data;
    uchar* lpV = V.data;


    Mat hsvImg;
    cvtColor(src, hsvImg, COLOR_BGR2HSV);


    // definire pointer la matricea (24 biti/pixeli) a imaginii HSV
    uchar* hsvDataPtr = hsvImg.data;


    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            int hi = i * width * 3 + j * 3;
            int gi = i * width + j;

```

```

        lpH[gi] = hsvDataPtr[hi] * 510 / 360;           // lpH = 0 .. 255
        lpS[gi] = hsvDataPtr[hi + 1];                 // lpS = 0 .. 255
        lpV[gi] = hsvDataPtr[hi + 2];                 // lpV = 0 .. 255
    }
}

imshow("input image", src);
imshow("H", H);
imshow("S", S);
imshow("V", V);

waitKey();
}
}

void testResize()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat src;
        src = imread(fname);
        Mat dst1, dst2;
        //without interpolation
        resizeImg(src, dst1, 320, false);
        //with interpolation
        resizeImg(src, dst2, 320, true);
        imshow("input image", src);
    }
}

```

```

        imshow("resized image (without interpolation)", dst1);
        imshow("resized image (with interpolation)", dst2);
        waitKey();
    }
}

```

```

void testCanny()
{
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        Mat src, dst, gauss;
        src = imread(fname, IMREAD_GRAYSCALE);
        double k = 0.4;
        int pH = 50;
        int pL = (int)k * pH;
        GaussianBlur(src, gauss, Size(5, 5), 0.8, 0.8);
        Canny(gauss, dst, pL, pH, 3);
        imshow("input image", src);
        imshow("canny", dst);
        waitKey();
    }
}

```

```

void testVideoSequence()
{
    VideoCapture cap("Videos/rubic.avi");
    if (!cap.isOpened()) {
        printf("Cannot open video capture device.\n");
    }
}

```

```

        waitKey(0);
        return;
    }

    Mat edges;
    Mat frame;
    char c;

    while (cap.read(frame))
    {
        Mat grayFrame;
        cvtColor(frame, grayFrame, COLOR_BGR2GRAY);
        Canny(grayFrame, edges, 40, 100, 3);
        imshow("source", frame);
        imshow("gray", grayFrame);
        imshow("edges", edges);
        c = waitKey(0); // waits a key press to advance to the next frame
        if (c == 27) {
            // press ESC to exit
            printf("ESC pressed - capture finished\n");
            break; //ESC pressed
        };
    }
}

void testSnap()
{
    VideoCapture cap(0); // open the default camera (i.e. the built in web cam)

```

```

if (!cap.isOpened()) // openenig the video device failed
{
    printf("Cannot open video capture device.\n");
    return;
}

Mat frame;

char numberStr[256];
char fileName[256];

// video resolution
Size capS = Size((int)cap.get(CAP_PROP_FRAME_WIDTH),
    (int)cap.get(CAP_PROP_FRAME_HEIGHT));

// Display window
const char* WIN_SRC = "Src"; //window for the source frame
namedWindow(WIN_SRC, WINDOW_AUTOSIZE);
moveWindow(WIN_SRC, 0, 0);

const char* WIN_DST = "Snapped"; //window for showing the snapped frame
namedWindow(WIN_DST, WINDOW_AUTOSIZE);
moveWindow(WIN_DST, capS.width + 10, 0);

char c;

int frameNum = -1;

int frameCount = 0;

for (;;)
{

```

```

cap >> frame; // get a new frame from camera
if (frame.empty())
{
    printf("End of the video file\n");
    break;
}

++frameNum;

imshow(WIN_SRC, frame);

c = waitKey(10); // waits a key press to advance to the next frame
if (c == 27) {
    // press ESC to exit
    printf("ESC pressed - capture finished");
    break; //ESC pressed
}

if (c == 115) { //'s' pressed - snapp the image to a file
    frameCount++;
    fileName[0] = NULL;
    sprintf(numberStr, "%d", frameCount);
    strcat(fileName, "Images/A");
    strcat(fileName, numberStr);
    strcat(fileName, ".bmp");
    bool bSuccess = imwrite(fileName, frame);
    if (!bSuccess)
    {
        printf("Error writing the snapped image\n");
    }
}

```

```

        else
            imshow(WIN_DST, frame);
    }
}

}

```

```

void MyCallBackFunc(int event, int x, int y, int flags, void* param)
{
    Mat* src = (Mat*)param;
    if (event == EVENT_LBUTTONDOWN)
    {
        printf("Pos(x,y): %d,%d  Color(RGB): %d,%d,%d\n",
            x, y,
            (int)(*src).at<Vec3b>(y, x)[2],
            (int)(*src).at<Vec3b>(y, x)[1],
            (int)(*src).at<Vec3b>(y, x)[0]);
    }
}

```

```

void testMouseClicked()
{
    Mat src;
    // Read image from file
    char fname[MAX_PATH];
    while (openFileDialog(fname))
    {
        src = imread(fname);
        //Create a window
    }
}

```

```

        namedWindow("My Window", 1);

        //set the callback function for any mouse event
        setMouseCallback("My Window", MyCallBackFunc, &src);

        //show the image
        imshow("My Window", src);

        // Wait until user press some key
        waitKey(0);
    }
}

//-----lab7-----///

```

```

void dilatare() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = 255;
        }
    }

    int dj[4] = { -1,-1,-1,0 };
    int di[4] = { -1,0,1,-1 };

```

```

int ni, nj;

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (img.at<uchar>(i, j) == 0) {
            dst.at<uchar>(i, j) = 0;
            for (int k = 0; k < 4; k++) {
                //for each neighbour
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    dst.at<uchar>(ni, nj) = 0;
                }
            }
        }
    }
}

imshow("Initial", img);
imshow("Dilatare Output", dst);
waitKey(0);

}

void dilatare(int n) {
    char fname[MAX_PATH];
    openFileDlg(fname);

```

```

Mat img = imread(fname, IMREAD_GRAYSCALE);
Mat dst[100];

for (int i = 0; i < n + 1; i++) {
    dst[i] = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
}

dst[0] = img;

int dj[4] = { -1,-1,-1,0 };
int di[4] = { -1,0,1,-1 };

int ni, nj;

for (int l = 0; l < n; l++) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            if (dst[l].at<uchar>(i, j) == 0) {
                for (int k = 0; k < 4; k++) {
                    ni = i + di[k];
                    nj = j + dj[k];
                    if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0)
{
                        dst[l + 1].at<uchar>(ni, nj) = 0;
                    }
                }
            }
        }
    }
}

```

```

    }

    imshow("Initial", img);
    imshow("Dilatare", dst[n]);
    waitKey(0);

}

void eroziune() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = 255;
        }
    }

    int dj[4] = { -1,-1,-1,0 };
    int di[4] = { -1,0,1,-1 };

    int ni, nj;
    int nrN;

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            nrN = 0;

```

```

        if (img.at<uchar>(i, j) == 0) {
            for (int k = 0; k < 4; k++) {
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    if (img.at<uchar>(ni, nj) == 0) {
                        nrN++;
                    }
                }
            }

            if (nrN == 4) {
                dst.at<uchar>(i, j) = 0;
            }
            else {
                dst.at<uchar>(i, j) = 255;
            }
        }
    }

    imshow("Initial", img);
    imshow("Eroziune Output", dst);
    waitKey(0);
}

void eroziune(int n) {
    char fname[MAX_PATH];

```

```

openFileDialog(fname);

Mat img = imread(fname, IMREAD_GRAYSCALE);

Mat dst[100];

for (int i = 0; i < n + 1; i++) {
    dst[i] = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
}

dst[0] = img;

int dj[4] = { -1,-1,-1,0 };
int di[4] = { -1,0,1,-1 };

int ni, nj, nrN;

for (int l = 0; l < n; l++) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            nrN = 0;
            if (img.at<uchar>(i, j) == 0) {
                for (int k = 0; k < 4; k++) {
                    ni = i + di[k];
                    nj = j + dj[k];
                    if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0)
{
                        if (dst[l].at<uchar>(ni, nj) == 0) {
                            nrN++;
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
    }

    }

    if (nrN == 4) {
        dst[l + 1].at<uchar>(i, j) = 0;
    }
    else {
        dst[l + 1].at<uchar>(i, j) = 255;
    }
}

}

}

```

```

imshow("Initial", img);
imshow("Eroziune", dst[n]);
waitKey(0);

```

```

}

```

```

void inchidere() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);
    Mat dst_final = Mat(img.rows, img.cols, CV_8UC1);
}

```

```

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        dst.at<uchar>(i, j) = 255;
        dst_final.at<uchar>(i, j) = 255;
    }
}

```

```
int dj[4] = { -1,-1,-1,0 };
```

```
int di[4] = { -1,0,1,-1 };
```

```
int ni, nj;
```

```

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (img.at<uchar>(i, j) == 0) {
            dst.at<uchar>(i, j) = 0;
            for (int k = 0; k < 4; k++) {
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    dst.at<uchar>(ni, nj) = 0;
                }
            }
        }
    }
}

```

```
int nrN;
```

```

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        nrN = 0;
        if (dst.at<uchar>(i, j) == 0) {
            for (int k = 0; k < 4; k++) {
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    if (dst.at<uchar>(ni, nj) == 0) {
                        nrN++;
                    }
                }
            }
        }
        if (nrN == 4) {
            dst_final.at<uchar>(i, j) = 0;
        }
        else {
            dst_final.at<uchar>(i, j) = 255;
        }
    }
}

imshow("Initial", img);
imshow("Inchidere Output", dst_final);
waitKey(0);
}

```

```

void inchidere(int n) {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst_final[100];

    Mat dst[100];

    for (int i = 0; i < n + 1; i++) {
        dst[i] = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
        dst_final[i] = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    }

    int dj[4] = { -1,-1,-1,0 };
    int di[4] = { -1,0,1,-1 };

    int ni, nj, nrN;

    for (int l = 0; l < n; l++)
        for (int i = 0; i < img.rows; i++) {
            for (int j = 0; j < img.cols; j++) {
                if (img.at<uchar>(i, j) == 0) {
                    for (int k = 0; k < 4; k++) {
                        ni = i + di[k];
                        nj = j + dj[k];
                        if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0)
                            dst[l + 1].at<uchar>(ni, nj) = 0;
                    }
                }
            }
        }
}

```

```

    }
    }
    }
    }
    }

dst_final[0] = dst[n];

for (int l = 0; l < n; l++) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            nrN = 0;
            if (dst[n].at<uchar>(i, j) == 0) {
                for (int k = 0; k < 4; k++) {
                    ni = i + di[k];
                    nj = j + dj[k];
                    if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0)
{
                        if (dst_final[l].at<uchar>(ni, nj) == 0) {
                            nrN++;
                        }
                    }
                }
            }
        }
    }

    if (nrN == 4) {
        dst_final[l + 1].at<uchar>(i, j) = 0;
    }
}

```

```

        else {
            dst_final[l + 1].at<uchar>(i, j) = 255;
        }
    }
}

imshow("Inchidere Final", dst_final[n]);
waitKey(0);
}

```

```

void deschidere() {
    char fname[MAX_PATH];
    openFileDialog(fname);

    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);
    Mat dst_final = Mat(img.rows, img.cols, CV_8UC1);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = 255;
            dst_final.at<uchar>(i, j) = 255;
        }
    }

    int dj[4] = { -1,-1,-1,0 };
    int di[4] = { -1,0,1,-1 };

    int ni, nj;
}

```

```
int nrN;
```

```
for (int i = 0; i < img.rows; i++) {  
    for (int j = 0; j < img.cols; j++) {  
        nrN = 0;  
        if (img.at<uchar>(i, j) == 0) {  
            for (int k = 0; k < 4; k++) {  
                ni = i + di[k];  
                nj = j + dj[k];  
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {  
                    if (img.at<uchar>(ni, nj) == 0) {  
                        nrN++;  
                    }  
                }  
            }  
        }  
        if (nrN == 4) {  
            dst.at<uchar>(i, j) = 0;  
        }  
        else {  
            dst.at<uchar>(i, j) = 255;  
        }  
    }  
}
```

```
for (int i = 0; i < img.rows; i++) {  
    for (int j = 0; j < img.cols; j++) {
```

```

        if (dst.at<uchar>(i, j) == 0) {
            for (int k = 0; k < 4; k++) {
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    dst_final.at<uchar>(ni, nj) = 0;
                }
            }
        }
    }
}

imshow("Initial", img);
imshow("Intermediar", dst);
imshow("Deschidere Output", dst_final);
waitKey(0);
}

```

```

void extragereContur() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat rez = Mat(img.rows, img.cols, CV_8UC1);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = 255;

```

```

        rez.at<uchar>(i, j) = 255;
    }
}

int dj[4] = { -1,-1,-1,0 };
int di[4] = { -1,0,1,-1 };

int ni, nj;
int nrN;

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        nrN = 0;
        if (img.at<uchar>(i, j) == 0) {
            for (int k = 0; k < 4; k++) {
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0) {
                    if (img.at<uchar>(ni, nj) == 0) {
                        nrN++;
                    }
                }
            }
        }
    }

    if (nrN == 4) {
        dst.at<uchar>(i, j) = 0;
    }
}

```

```

        else {
            dst.at<uchar>(i, j) = 255;
        }
    }
}

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (img.at<uchar>(i, j) != dst.at<uchar>(i, j)) {
            rez.at<uchar>(i, j) = 0;
        }
    }
}

imshow("Initial", img);
imshow("Contur Output", rez);
waitKey(0);

}

bool diferit(Mat m1, Mat m2) {
    for (int i = 0; i < m1.rows; i++) {
        for (int j = 0; j < m1.cols; j++) {
            if (m1.at<uchar>(i, j) != m2.at<uchar>(i, j)) {
                return true;
            }
        }
    }
}

```

```

        return false;
    }

    void copy(Mat m1, Mat m2) {
        for (int i = 0; i < m1.rows; i++) {
            for (int j = 0; j < m1.cols; j++) {
                m1.at<uchar>(i, j) = m2.at<uchar>(i, j);
            }
        }
    }
}

```

```

void umplereRegiuni() {
    char fname[MAX_PATH];
    openFileDialog(fname);

    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat comp = Mat(img.rows, img.cols, CV_8UC1);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1);
    Mat dst_aux = Mat(img.rows, img.cols, CV_8UC1);
    Mat rez_dilatare = Mat(img.rows, img.cols, CV_8UC1);
    Mat rez_intersectie = Mat(img.rows, img.cols, CV_8UC1);
    Mat rez = Mat(img.rows, img.cols, CV_8UC1);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            comp.at<uchar>(i, j) = 255;
            dst.at<uchar>(i, j) = 255;
            rez_dilatare.at<uchar>(i, j) = 255;
            rez_intersectie.at<uchar>(i, j) = 255;
        }
    }
}

```

```

        rez.at<uchar>(i, j) = 255;
    }
}

int dj[4] = { -1,-1,-1,0 };
int di[4] = { -1,0,1,-1 };

int ni, nj;

dst.at<uchar>(int(img.rows / 2), int(img.cols / 2)) = 0;

//complement

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (img.at<uchar>(i, j) == 0) {
            comp.at<uchar>(i, j) = 255;
        }
        else {
            comp.at<uchar>(i, j) = 0;
        }
    }
}

do {

    copy(dst_aux, dst);

    //dilatate dst_aux

```

```

for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (dst_aux.at<uchar>(i, j) == 0) {
            rez_dilatare.at<uchar>(i, j) = 0;
            for (int k = 0; k < 4; k++) {
                //for each neighbour
                ni = i + di[k];
                nj = j + dj[k];
                if (ni < img.rows && ni >= 0 && nj < img.cols && nj >= 0)

                    rez_dilatare.at<uchar>(ni, nj) = 0;
            }
        }
    }
}

//intersection
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (rez_dilatare.at<uchar>(i, j) == 0 && comp.at<uchar>(i, j) == 0) {
            rez_intersectie.at<uchar>(i, j) = 0;
        }
    }
}

copy(dst, rez_intersectie);

} while (diferit(dst, dst_aux));

```

```

//reuniune
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (dst.at<uchar>(i, j) == 0 || img.at<uchar>(i, j) == 0) {
            rez.at<uchar>(i, j) = 0;
        }
        else {
            rez.at<uchar>(i, j) = 255;
        }
    }
}

imshow("Initial", img);
imshow("Umplere Output", rez);
waitKey(0);
}

```

//laborator saptamana 8

```

void medie(Mat img, float* medie) {
    float suma = 0;

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            suma += img.at<uchar>(i, j);
        }
    }

    *(medie) = (1.0 / (img.rows * img.cols)) * suma;
}

```

```

        //printf("Medie: %f\n", medie);
    }

void deviatie(Mat img, float medie, float* deviatie) {
    float suma = 0;
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            suma += (float)(img.at<uchar>(i, j) - medie) * (float)(img.at<uchar>(i, j) -
medie);
        }
    }

    *(deviatie) = sqrt((1.0 / (img.cols * img.rows)) * suma);
}

void calcHistograma(Mat img, int* hist) {
    int height = img.rows;
    int width = img.cols;

    for (int i = 0; i < 255; i++) {
        hist[i] = 0;
    }
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            int index = img.at<uchar>(i, j);
            hist[index] = hist[index] + 1;
        }
    }
}

```

```

void calcHistogramaCumulativa(int* hist, int* hist_c) {
    for (int i = 0; i < 255; i++) {
        hist_c[i] = 0;
    }

    hist_c[0] = hist[0];

    for (int i = 1; i < 256; i++)
    {
        hist_c[i] = hist_c[i - 1] + hist[i];
    }
}

```

// Media, deviatia standard, histograma si histograma cumulativa

```

void ex1Lab8() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    int* hist = (int*)calloc(sizeof(int), 256);
    int* hist_c = (int*)calloc(sizeof(int), 256);
    float medie_img, deviatie_img, prag;
    medie(img, &medie_img);
    deviatie(img, medie_img, &deviatie_img);
    printf("Medie: %f\n", medie_img);
    printf("Deviatie standard: %f\n", deviatie_img);
    calcHistograma(img, hist);
    calcHistogramaCumulativa(hist, hist_c);
}

```

```

        showHistogram("Histograma", hist, 400, 500);

        showHistogram("Histograma Cumulativa", hist_c, 400, 500);

        waitKey(0);
    }

void determinarePragBinarizare(Mat img, int* hist, float eroare, float* prag) {
    int minim = 0;
    int maxim = 255;

    while (hist[minim] == 0)
        minim++;

    while (hist[maxim] == 0)
        maxim--;

    float tk = (maxim + minim) / 2.0;

    float medieG1 = 0, medieG2 = 0, tkminus1;

    float n1 = 0, n2 = 0, produs1 = 0, produs2 = 0;

    do {
        tkminus1 = tk;
        for (int i = minim; i <= tk; i++) {
            n1 += hist[i];
            produs1 += i * hist[i];
        }
    }

```

```

        for (int i = tk + 1; i <= maxim; i++) {
            n2 += hist[i];
            produs2 += i * hist[i];
        }

        medieG1 = (1.0 / n1) * produs1;
        medieG2 = (1.0 / n2) * produs2;

        tk = (medieG1 + medieG2) / 2.0;
    } while (tk - tkminus1 < eroare);

    *(prag) = tk;
}

void binarizare(Mat img, float prag, Mat dst) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            if (img.at<uchar>(i, j) < prag) {
                dst.at<uchar>(i, j) = 0;
            }
        }
    }
}

void ex2Lab8() {
    char fname[MAX_PATH];
    openFileDialog(fname);

    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
}

```

```

int* hist = (int*)calloc(sizeof(int), 256);

calcHistograma(img, hist);

float prag;

determinarePragBinarizare(img, hist, 0.1, &prag);


binarizare(img, prag, dst);

imshow("Imagine", img);

imshow("Binarizare", dst);

waitKey(0);

}

```

```

void identitate(Mat img, Mat dst) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = img.at<uchar>(i, j);
        }
    }
}

```

```

void cleanImg(Mat img) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            img.at<uchar>(i, j) = 255;
        }
    }
}

```

```

void negativ(Mat img, Mat dst) {
    for (int i = 0; i < img.rows; i++) {

```

```

        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = 255 - img.at<uchar>(i, j);
        }
    }
}

```

```

void luminiozitate(Mat img, Mat dst, int offset) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            int val = img.at<uchar>(i, j) + offset;
            if (val <= 255 && val >= 0) {
                dst.at<uchar>(i, j) = val;
            }
        }
    }
}

```

```

void contrast(Mat img, Mat dst, int minimIn, int maximOut, int* hist) {
    int minimIn = 0;
    int maximIn = 255;

    while (hist[minimIn] == 0)
        minimIn++;

    while (hist[maximIn] == 0)
        maximIn--;

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {

```

```

                dst.at<uchar>(i, j) = minimOut + (float)(img.at<uchar>(i, j) - minimIn) *
((float)(maximOut - minimOut) / (float)(maximIn - minimIn));
            }
        }
    }
}

```

```

void corectie_gamma(Mat img, Mat dst, float gamma) {
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            float val = 255.0 * pow(((float)img.at<uchar>(i, j) / 255.0), gamma);
            if (val >= 0 && val <= 255) {
                dst.at<uchar>(i, j) = val;
            }
        }
    }
}

```

```

void ex3Lab8() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    identitate(img, dst);
    imshow("Imagine", img);
    imshow("Identitate", dst);
    waitKey(0);
    cleanImg(dst);
    negativ(img, dst);
    imshow("Imagine", img);
}

```

```
    imshow("Negativ", dst);  
    waitKey(0);  
    cleanImg(dst);  
    luminiozitate(img, dst, 50);  
    imshow("Imagine", img);  
    imshow("Luminiozitate", dst);  
    waitKey(0);  
    cleanImg(dst);  
    int* hist = (int*)calloc(sizeof(int), 256);  
    int* hist_out = (int*)calloc(sizeof(int), 256);  
    calcHistograma(img, hist);  
    contrast(img, dst, 10, 250, hist);  
    calcHistograma(dst, hist_out);  
    imshow("Imagine", img);  
    showHistogram("Initial", hist, 400, 500);  
    imshow("Contrast", dst);  
    showHistogram("Final", hist_out, 400, 500);  
    waitKey(0);  
    cleanImg(dst);  
    corectie_gamma(img, dst, 0.5);  
    imshow("Imagine", img);  
    imshow("Gamma", dst);  
    waitKey(0);  
}
```

```
void egalHist(Mat img, Mat dst) {  
    int* hist = (int*)calloc(sizeof(int), 256);  
    int* hist_c = (int*)calloc(sizeof(int), 256);
```

```

    calcHistograma(img, hist);
    calcHistogramaCumulativa(hist, hist_c);

    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            dst.at<uchar>(i, j) = (float)255 / (img.rows * img.cols) * hist_c[img.at<uchar>(i,
j)];
        }
    }
}

```

```

void ex4Lab8() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat img = imread(fname, IMREAD_GRAYSCALE);
    Mat dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    int* hist = (int*)calloc(sizeof(int), 256);
    egalHist(img, dst);
    calcHistograma(dst, hist);
    showHistogram("Histograma Egalata", hist, 400, 500);
    imshow("Imagine", img);
    imshow("Egalizare Histograma", dst);
    waitKey(0);
}

```

//lab9

```

void sumPosMat_int(Mat m, int* sum) {
    *(sum) = 0;
    for (int i = 0; i < m.rows; i++) {
        for (int j = 0; j < m.cols; j++) {

```

```

        if (m.at<int>(i, j) >= 0) {
            *(sum) += m.at<int>(i, j);
        }
    }
}

void sumAbsNegMat_int(Mat m, int* sum) {
    *(sum) = 0;
    for (int i = 0; i < m.rows; i++) {
        for (int j = 0; j < m.cols; j++) {
            if (m.at<int>(i, j) < 0) {
                *(sum) += abs(m.at<int>(i, j));
            }
        }
    }
}

```

```

void sumPosMat(Mat m, float* sum) {
    *(sum) = 0;
    for (int i = 0; i < m.rows; i++) {
        for (int j = 0; j < m.cols; j++) {
            if (m.at<float>(i, j) >= 0) {
                *(sum) += m.at<float>(i, j);
            }
        }
    }
}

```

```

void sumAbsNegMat(Mat m, float* sum) {
    *(sum) = 0;

```

```

for (int i = 0; i < m.rows; i++) {
    for (int j = 0; j < m.cols; j++) {
        if (m.at<float>(i, j) < 0) {
            *(sum) += abs(m.at<float>(i, j));
        }
    }
}
}

```

```

boolean isInside(Mat* img, int row, int col) {
    if (row < img->rows && col < img->cols && row >= 0 && col >= 0) {
        return true;
    }
    return false;
}

```

```

void ordonare(int* arr, int* mid, int len) {
    sort(arr, arr + len);
    *(mid) = arr[len / 2];
}

```

```

void convolution(Mat& filter, Mat_<uchar>& img, Mat_<uchar>& output) {

```

```

    output.create(img.size());
    memcpy(output.data, img.data, img.rows * img.cols * sizeof(uchar));

```

```

    float scalingCoeff = 1;
    float additionFactor = 0;

```

```

    //TODO: decide if the filter is low pass or high pass and compute the scaling coefficient and the
    addition factor

```

```

// low pass if all elements >= 0
// high pass has elements < 0
bool low = true;
for (int i = 0; i < filter.rows; i++) {
    for (int j = 0; j < filter.cols; j++) {
        if (filter.at<float>(i, j) < 0) {
            low = false;
        }
    }
}

// compute scaling coefficient and addition factor for low pass and high pass
// low pass: additionFactor = 0, scalingCoeff = sum of all elements
// high pass: formula 9.20

if (low) {
    additionFactor = 0;
    sumPosMat(filter, &scalingCoeff);
    //printf("Suma : %d\n", scalingCoeff);
}
else {
    additionFactor = 255 / 2;
    float splus = 0, sminus = 0;
    sumPosMat(filter, &splus);
    sumAbsNegMat(filter, &sminus);
    scalingCoeff = 2 * max(splus, sminus);
}

// TODO: implement convolution operation (formula 9.2)

```

// do not forget to divide with the scaling factor and add the addition factor in order to have values between [0, 255]

```
int k = (filter.rows - 1) / 2;

for (int i = k; i < img.rows - k; i++) {
    for (int j = k; j < img.cols - k; j++) {
        float res = 0; // (9.2) dreapta formulei
        for (int u = 0; u < filter.rows; u++) {
            for (int v = 0; v < filter.cols; v++) {
                if (isInside(&img, i + u - k, j + v - k)) {
                    res += filter.at<float>(u, v) * img.at<uchar>(i + u - k, j + v
- k);
                }
            }
        }
        if (res > 255) {
            res = 255;
        }
        if (res < 0) {
            res = 0;
        }
        res /= scalingCoeff;
        res += additionFactor;
        output.at<uchar>(i, j) = res;
    }
}
```

//ex1 lab 10

```

void filtru_median() {
    double t = (double)getTickCount();

    int w;

    cout << "Introduceti valoarea lui w. (3,5,7)\n";

    cin >> w;

    char fname[MAX_PATH];

    openFileDialog(fname);

    Mat_<uchar> img = imread(fname, IMREAD_GRAYSCALE);

    Mat dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));

    identitate(img, dst);

    for (int i = w; i < img.rows - w; i++) {
        for (int j = w; j < img.cols - w; j++) {
            int* vals = (int*)calloc(sizeof(int), w * w);

            int k = 0;

            for (int u = 0; u < w; u++) {
                for (int v = 0; v < w; v++) {
                    vals[k] = img.at<uchar>(i + u - w, j + v - w);

                    k++;
                }
            }

            int mid = 0;

            ordonare(vals, &mid, w * w);

            dst.at<uchar>(i, j) = mid;

            k = 0;
        }
    }

    t = ((double)getTickCount() - t) / getTickFrequency();
}

```

```

printf("Timp: %.3f [ms] \n", t * 1000);

imshow("Initial", img);

imshow("Final", dst);

waitKey(0);
}

//ex2 lab 10

void filtru_gaussian() {
    double t = (double)getTickCount();

    int w;
    cout << "Introduceti valoarea lui w. (3,5,7)\n";
    cin >> w;
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat_<uchar> img = imread(fname, IMREAD_GRAYSCALE);
    Mat_<uchar> dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    float sigma = (float)w / 6.0;
    Mat g = Mat(w, w, CV_32FC1);

    float ct = 1.0 / (2 * PI * sigma * sigma);
    for (int i = 0; i < w; i++) {
        for (int j = 0; j < w; j++) {
            g.at<float>(i, j) = ct * exp(-((i - w / 2) * (i - w / 2) + (j - w / 2) * (j - w / 2))) / (2 *
sigma * sigma));
        }
    }
}

```

```

    }

    convolution(g, img, dst);

    t = ((double)getTickCount() - t) / getTickFrequency();

    printf("Timp: %.3f [ms] \n", t * 1000);

    imshow("Initial", img);

    imshow("Final", dst);

    waitKey(0);
}

Mat convolutieGrad(Mat src, Mat kernel) {

    float S = 0;

    float k = 0;

    Mat dst = Mat(src.size(), CV_8UC1);
    Mat dst1 = Mat(src.size(), CV_8UC1);

    for (int i = 0; i < kernel.rows; i++)
        for (int j = 0; j < kernel.cols; j++)
            printf("%f\n", kernel.at<float>(i, j));

    k = kernel.cols / 2;

    if (k < 2) k = kernel.rows / 2;

    dst.convertTo(dst, CV_32FC1);

```

```

for (int i = k; i < src.rows - k; i++) {
    for (int j = k; j < src.cols - k; j++) {

        //parcurgem kernelul din jurul fiecarui pixel
        S = 0.0;
        for (int ki = 0; ki < kernel.rows; ki++)
            for (int kj = 0; kj < kernel.cols; kj++) {

                //printf("pixelul %f * kernel elem. %f la %d %d\n", 1.0 *
src.at<uchar>(i + ki, j + kj), kernel.at<float>(ki + 1, kj + 1), ki, kj);

                S += src.at<byte>(i + ki - k, j + kj - k) * kernel.at<float>(ki, kj);

            }

        dst.at<float>(i, j) = S / (4 * sqrt(2));

    }
    //getchar();
}

dst.convertTo(dst, CV_8UC1);

return dst;

}

```

```
Mat convolutieNoNorm(Mat src, Mat kernel) {
```

```
    float S = 0;
```

```
    float k = 0;
```

```
    Mat dst = Mat(src.size(), CV_8UC1);
```

```
    Mat dst1 = Mat(src.size(), CV_8UC1);
```

```
    for (int i = 0; i < kernel.rows; i++)
```

```
        for (int j = 0; j < kernel.cols; j++)
```

```
            printf("%f\n", kernel.at<float>(i, j));
```

```
    k = kernel.cols / 2;
```

```
    if (k < 2) k = kernel.rows / 2;
```

```
    dst.convertTo(dst, CV_32FC1);
```

```
    for (int i = k; i < src.rows - k; i++) {
```

```
        for (int j = k; j < src.cols - k; j++) {
```

```
            //parcurem kernelul din jurul fiecarui pixel
```

```
            S = 0.0;
```

```
            for (int ki = 0; ki < kernel.rows; ki++)
```

```
                for (int kj = 0; kj < kernel.cols; kj++) {
```

```

//printf("pixelul %f * kernel elem. %f la %d %d\n", 1.0 *
src.at<uchar>(i + ki, j + kj), kernel.at<float>(ki + 1, kj + 1), ki, kj);

S += src.at<byte>(i + ki - k, j + kj - k) * kernel.at<float>(ki, kj);

}

```

```

dst.at<float>(i, j) = S / (4 * sqrt(2));

```

```

}
//getchar();
}

```

```

//dst.convertTo(dst, CV_8UC1);

```

```

return dst;

```

```

}

```

```

//lab11-1

```

```

void gradFX() {

```

```

    Mat src, dst;

```

```

    char fname[MAX_PATH];

```

```

    int n = 3;

```

```

    Mat kernel = Mat(n, n, CV_32F);

```

```
while (openFileDialog(fname)) {  
  
    src = imread(fname, IMREAD_GRAYSCALE);  
  
    kernel.at<float>(0, 0) = -1;  
    kernel.at<float>(1, 0) = -2;  
    kernel.at<float>(2, 0) = -1;  
    kernel.at<float>(0, 1) = 0;  
    kernel.at<float>(1, 1) = 0;  
    kernel.at<float>(2, 1) = 0;  
    kernel.at<float>(0, 2) = 1;  
    kernel.at<float>(1, 2) = 2;  
    kernel.at<float>(2, 2) = 1;  
  
    dst = convolutieGrad(src, kernel);  
  
    imshow("src", src);  
    imshow("dst", dst);  
  
    waitKey();  
}  
  
}
```

```
void gradFY() {  
  
    Mat src, dst;  
    char fname[MAX_PATH];  
  
    int n = 3;  
  
    Mat kernel = Mat(n, n, CV_32F);  
  
    while (openFileDialog(fname)) {  
  
        src = imread(fname, IMREAD_GRAYSCALE);  
  
        kernel.at<float>(0, 0) = 1;  
        kernel.at<float>(1, 0) = 0;  
        kernel.at<float>(2, 0) = -1;  
        kernel.at<float>(0, 1) = 2;  
        kernel.at<float>(1, 1) = 0;  
        kernel.at<float>(2, 1) = -2;  
        kernel.at<float>(0, 2) = 1;  
        kernel.at<float>(1, 2) = 0;  
        kernel.at<float>(2, 2) = -1;  
  
        dst = convolutieGrad(src, kernel);  
    }  
}
```

```

        imshow("src", src);
        imshow("dst", dst);

        waitKey();
    }

}

```

```

void modulSiDirectie() {

```

```

    Mat src, modul, directie, dstX, dstY;
    char fname[MAX_PATH];

```

```

    int n = 3;

```

```

    Mat kernel = Mat(n, n, CV_32F);

```

```

    while (openFileDialog(fname)) {

```

```

        src = imread(fname, IMREAD_GRAYSCALE);

```

```

        kernel.at<float>(0, 0) = -1;

```

```

        kernel.at<float>(1, 0) = -2;

```

```

        kernel.at<float>(2, 0) = -1;

```

```

        kernel.at<float>(0, 1) = 0;

```

```

        kernel.at<float>(1, 1) = 0;

```

```
kernel.at<float>(2, 1) = 0;
kernel.at<float>(0, 2) = 1;
kernel.at<float>(1, 2) = 2;
kernel.at<float>(2, 2) = 1;
```

```
dstX = convolutieNoNorm(src, kernel);
```

```
kernel.at<float>(0, 0) = 1;
kernel.at<float>(1, 0) = 0;
kernel.at<float>(2, 0) = -1;
kernel.at<float>(0, 1) = 2;
kernel.at<float>(1, 1) = 0;
kernel.at<float>(2, 1) = -2;
kernel.at<float>(0, 2) = 1;
kernel.at<float>(1, 2) = 0;
kernel.at<float>(2, 2) = -1;
```

```
dstY = convolutieNoNorm(src, kernel);
```

```
modul = Mat(src.size(), CV_32FC1);
directie = Mat(src.size(), CV_32FC1);
```

```
for (int i = 0; i < src.rows; i++)
    for (int j = 0; j < src.cols; j++) {
```

```
        modul.at<float>(i, j) = sqrt(dstX.at<float>(i, j) * dstX.at<float>(i, j) +
dstY.at<float>(i, j) * dstY.at<float>(i, j));
```

```
        directie.at<float>(i, j) = (180. / PI) * atan2(dstX.at<float>(i, j),  
dstY.at<float>(i, j));
```

```
    }
```

```
    modul.convertTo(modul, CV_8UC1);
```

```
    directie.convertTo(directie, CV_8UC1);
```

```
    imshow("src", src);
```

```
    imshow("dstX", dstX);
```

```
    imshow("dstY", dstY);
```

```
    imshow("modul", modul);
```

```
    imshow("directie", directie);
```

```
    waitKey();
```

```
    }
```

```
}
```

```
//lab 12
```

```
void gaussian_filter(Mat_<uchar> img, Mat_<uchar> dst) {
```

```
    int w = 3;
```

```
    float sigma = (float)w / 6.0;
```

```
    Mat g = Mat(w, w, CV_32FC1);
```

```
    float ct = 1.0 / (2 * PI * sigma * sigma);
```

```

    for (int i = 0; i < w; i++) {
        for (int j = 0; j < w; j++) {
            g.at<float>(i, j) = ct * exp(((i - w / 2) * (i - w / 2) + (j - w / 2) * (j - w / 2))) / (2 *
sigma * sigma));
        }
    }

    convolution(g, img, dst);

}

```

```

void gradient_and_direction(Mat_<uchar> img, Mat sobel, Mat phi, Mat mag) {

    int fx_values[9] = { -1,-0,1,-2,0,2,-1,0,1 };
    Mat_<int> fx = Mat_<int>(3, 3, fx_values);
    int fy_values[9] = { 1,2,1,0,0,0,-1,-2,-1 };
    Mat_<int> fy = Mat_<int>(3, 3, fy_values);

    Mat_<float> x, y;

    Mat Gx = Mat(img.rows, img.cols, CV_32FC1);
    Mat Gy = Mat(img.rows, img.cols, CV_32FC1);

    for (int i = 1; i < img.rows - 1; i++)
    {
        for (int j = 1; j < img.cols - 1; j++)
        {
            Gx.at<float>(i, j) = 0;
            Gy.at<float>(i, j) = 0;

```

```

        for (int u = 0; u < 3; u++)
        {
            for (int v = 0; v < 3; v++)
            {
                Gx.at<float>(i, j) += fx.at<int>(u, v) * img.at<uchar>(i - 1 + u, j - 1
+ v);

                Gy.at<float>(i, j) += fy.at<int>(u, v) * img.at<uchar>(i - 1 + u, j - 1
+ v);

            }
        }

        mag.at<float>(i, j) = (float)sqrt(pow(Gx.at<float>(i, j), 2) + (pow(Gy.at<float>(i, j),
2))) / 4 * sqrt(2);

        phi.at<float>(i, j) = atan2(Gy.at<float>(i, j), Gx.at<float>(i, j));
        sobel.at<uchar>(i, j) = (uchar)mag.at<float>(i, j);
    }
}
}

```

```

void suppression(Mat img, Mat dst, Mat phi, Mat mag) {
    for (int i = 1; i < img.rows - 1; i++) {
        for (int j = 1; j < img.cols - 1; j++) {
            //slice 0
            if ((phi.at<float>(i, j) >= -CV_PI / 8 && phi.at<float>(i, j) <= CV_PI / 8) ||
(phi.at<float>(i, j) >= 7 * CV_PI / 8) || (phi.at<float>(i, j) <= -7 * CV_PI / 8))
            {
                if (mag.at<float>(i, j) >= mag.at<float>(i, j - 1) && mag.at<float>(i, j) >=
mag.at<float>(i, j + 1))

                    dst.at<uchar>(i, j) = (uchar)mag.at<float>(i, j);
            }
        }
    }
}

```

```

else

    dst.at<uchar>(i, j) = 0;

}

//slice 1

else if ((phi.at<float>(i, j) >= CV_PI / 8 && phi.at<float>(i, j) <= 3 * CV_PI / 8) ||
(phi.at<float>(i, j) >= -7 * CV_PI / 8 && phi.at<float>(i, j) <= -5 * CV_PI / 8))
{
    if (mag.at<float>(i, j) >= mag.at<float>(i - 1, j + 1) && mag.at<float>(i, j)
    >= mag.at<float>(i + 1, j - 1))

        dst.at<uchar>(i, j) = (uchar)mag.at<float>(i, j);

    else

        dst.at<uchar>(i, j) = 0;

}

//slice 2

else if ((phi.at<float>(i, j) >= 3 * CV_PI / 8 && phi.at<float>(i, j) <= 5 * CV_PI / 8)
|| (phi.at<float>(i, j) >= -5 * CV_PI / 8 && phi.at<float>(i, j) <= -3 * CV_PI / 8))
{
    if (mag.at<float>(i, j) >= mag.at<float>(i - 1, j) && mag.at<float>(i, j) >=
mag.at<float>(i + 1, j))

        dst.at<uchar>(i, j) = (uchar)mag.at<float>(i, j);

    else

        dst.at<uchar>(i, j) = 0;

}

//slice 3

else if ((phi.at<float>(i, j) >= 5 * CV_PI / 8 && phi.at<float>(i, j) <= 7 * CV_PI / 8)
|| (phi.at<float>(i, j) >= -3 * CV_PI / 8 && phi.at<float>(i, j) <= -CV_PI / 8))
{

```

```

        if (mag.at<float>(i, j) >= mag.at<float>(i - 1, j - 1) && mag.at<float>(i, j)
        >= mag.at<float>(i + 1, j + 1))
            dst.at<uchar>(i, j) = (uchar)mag.at<float>(i, j);
        else
            dst.at<uchar>(i, j) = 0;
    }
}
}
}

```

```

void adaptive_thresholding(Mat mag, Mat dst) {
    int* hist = (int*)calloc(sizeof(int), 256);
    calcHistograma(mag, hist);

    float p = 0.1;
    float k = 0.4;

    float nr_non_edge_pixels = (1 - p) * ((mag.rows - 2) * (mag.cols - 2) - hist[0]);
    float threshold_high = 0;

    int sum_hist = hist[1];
    for (int i = 2; i < 256; i++) {
        if (sum_hist >= nr_non_edge_pixels) {
            threshold_high = i;
            break;
        }
        sum_hist += hist[i];
    }
}

```

```

float threshold_low = k * threshold_high;

uchar NO_EDGE = 0, WEAK_EDGE = 128, STRONG_EDGE = 255;

for (int i = 0; i < mag.rows; i++) {
    for (int j = 0; j < mag.cols; j++) {
        if (mag.at<uchar>(i, j) < threshold_low) {
            dst.at<uchar>(i, j) = NO_EDGE;
        }
        if (mag.at<uchar>(i, j) >= threshold_low && mag.at<uchar>(i, j) <=
threshold_high) {
            dst.at<uchar>(i, j) = WEAK_EDGE;
        }
        if (mag.at<uchar>(i, j) > threshold_high) {
            dst.at<uchar>(i, j) = STRONG_EDGE;
        }
    }
}
}

```

```

void edge_linking(Mat adaptive, Mat canny) {

```

```

    identitate(adaptive, canny);

```

```

    uchar NO_EDGE = 0, WEAK_EDGE = 128, STRONG_EDGE = 255;

```

```

    queue<Point> q;

```

```

    int dj[8] = { -1, -1, -1, 0, 1, 1, 1, 0 };

```

```

int di[8] = { -1, 0, 1, 1, 1, 0, -1, -1 };

for (int i = 0; i < adaptive.rows; i++) {
    for (int j = 0; j < adaptive.cols; j++) {
        if (canny.at<uchar>(i, j) == STRONG_EDGE) {
            Point p = Point(i, j);
            q.push(p);
        }
        while (!q.empty()) {
            int x, y;
            Point first = q.front();
            q.pop();
            int ni, nj;
            for (int k = 0; k < 8; k++) {
                //for each neighbour
                ni = first.x + di[k];
                nj = first.y + dj[k];
                if (ni < adaptive.rows && ni >= 0 && nj < adaptive.cols && nj >=
0) {
                    if (canny.at<uchar>(ni, nj) == WEAK_EDGE) {
                        canny.at<uchar>(ni, nj) = STRONG_EDGE;
                        q.push({ ni,nj });
                    }
                }
            }
        }
    }
}
}

```

```

    for (int i = 0; i < adaptive.rows; i++) {
        for (int j = 0; j < adaptive.cols; j++) {
            if (canny.at<uchar>(i, j) == WEAK_EDGE) {
                canny.at<uchar>(i, j) = 0;
            }
        }
    }
}

void canny_method() {
    char fname[MAX_PATH];
    openFileDialog(fname);
    Mat_<uchar> img = imread(fname, IMREAD_GRAYSCALE);
    Mat_<uchar> dst_aux = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    Mat_<uchar> dst = Mat(img.rows, img.cols, CV_8UC1, Scalar(255, 255, 255));
    Mat phi = Mat(img.rows, img.cols, CV_32FC1);
    Mat mag = Mat(img.rows, img.cols, CV_32FC1);
    Mat supp = Mat(img.rows, img.cols, CV_8UC1);
    Mat adaptive = Mat(img.rows, img.cols, CV_8UC1);
    Mat canny = Mat(img.rows, img.cols, CV_8UC1);

    gaussian_filter(img, dst_aux);
    gradient_and_direction(dst_aux, dst, phi, mag);
    suppression(img, supp, phi, mag);
    adaptive_thresholding(supp, adaptive);
    edge_linking(adaptive, canny);
}

```

```

    imshow("Initial", img);

    imshow("Filtru Gaussian", dst_aux);

    imshow("Gradient", dst);

    imshow("Suppression", supp);

    imshow("Adaptive Thresholding", adaptive);

    imshow("Canny Edge", canny);

    waitKey(0);

}

```

```

int main()
{
    int op;

    int threshold; // for from grayscale to binary

    int i, j; // for test point inside

    do
    {
        system("cls");

        destroyAllWindows();

        printf("Menu:\n");

        printf(" 1 - Open image\n");

        printf(" 2 - Open BMP images from folder\n");

        printf(" 3 - Image negative - diblook style\n");

        printf(" 4 - BGR->HSV\n");

        printf(" 5 - Resize image\n");

        printf(" 6 - Canny edge detection\n");

        printf(" 7 - Edges in a video sequence\n");

        printf(" 8 - Snap frame from live video\n");

        printf(" 9 - Mouse callback demo\n");

        //-----lab1-----/
    }
}

```

```
printf(" 10 - Negative Image(1)\n");
printf(" 11 - Change Grey Level Multiplication(1)\n");
printf(" 12 - Create an image(1)\n");
printf(" 13 - Invers Matrix(1)\n");
//-----lab2-----/
printf(" 14 - From RGB to three grayscale images(2)\n");
printf(" 15 - From RGB to grayscale img(2)\n");
printf(" 16 - From grayscale to black and white/binary img(2)\n");
printf(" 17 - Compute the H, S and V values from the RGB(2)\n");
printf(" 18 - Point is inside(2)\n");
//-----lab3-----/
printf(" 19 - Compute Histogram and FDP then show(3)\n");
printf(" 20 - Multi Level Threshold(3)\n");
//-----lab4-----/
printf(" 21 - Objects Geometrical Features(4)\n");
//-----lab5-----/
printf(" 22 - BFS(5)\n");
//-----lab6-----/
printf(" 23 - Border tracing algorithm(6)\n");
printf(" 24 - Contour reconstruction(6)\n");
printf(" 25 - Dilatare\n");
printf(" 26 - Eroziune\n");
printf(" 27 - Inchidere\n");
printf(" 28 - Deschidere\n");
printf(" 29 - Extragerea conturului\n");
printf(" 30 - Umplerea regiunilor\n");
printf(" 31 - Media, deviatia standard, histograma si histograma cumulativa\n");
printf(" 32 - Determinarea automata a pragului de binarizare\n");
printf(" 33 - Functii transformare histograma\n");
```

```
printf(" 34 - Egalizarea histogramei\n");
printf(" 35 - Filtru median de dimensiune w\n");
printf(" 36 - Filtrare cu un nucleu gaussian bidimensional cu dimensiunea w.(3,5,7)\n");
printf(" 37 - Gradient fx \n");
printf(" 38 - Gradient fy \n");
printf(" 39 - Modul si Directie \n");
printf(" 40 - Canny \n");
printf(" 0 - Exit\n\n");
printf("Option: ");
scanf("%d", &op);
switch (op)
{
case 1:
    testOpenImage();
    break;
case 2:
    testOpenImagesFld();
    break;
case 3:
    testParcuregereSimplaDiblookStyle(); //diblook style
    break;
case 4:
    testBGR2HSV();
    break;
case 5:
    testResize();
    break;
case 6:
    testCanny();
```

```
        break;
case 7:
    testVideoSequence();
    break;
case 8:
    testSnap();
    break;
case 9:
    testMouseClicked();
    break;
case 10:
    negativeImage();
    break;
case 11:
    changeGreyLevelMul();
    break;
case 12:
    createImage();
    break;
case 13:
    create3x3InversMatrix();
    break;
case 14:
    fromUC3to3UC1();
    break;
case 15:
    fromRGBtoGrayscale();
    break;
case 16:
```

```
printf("Threshold:");  
scanf("%d", &threshold);  
fromGrayscaleToBW(threshold);  
break;
```

case 17:

```
computeHSVfromRGB();  
break;
```

case 18:

```
printf("i:");  
scanf("%d", &i);  
printf("j:");  
scanf("%d", &j);  
testIsPointInside(i, j);  
break;
```

case 19:

```
computeHistogramFDPandShow();  
break;
```

case 20:

```
multiLevelThreshold();  
break;
```

case 21:

```
testComputeGeometricalFeatures();  
break;
```

case 22:

```
douaTreceeri(); //bfs()  
break;
```

case 23:

```
borderTracing();  
break;
```

case 24:
 contourReconstruction();
 break;

case 25:
 dilatate(1);
 break;

case 26:
 eroziune(1);
 break;

case 27:
 inchidere();
 inchidere(1);
 break;

case 28:
 deschidere();
 break;

case 29:
 extragereContur();
 break;

case 30:
 umplereRegiuni();
 break;

case 31:
 ex1Lab8();
 break;

case 32:
 ex2Lab8();
 break;

case 33:

```

        ex3Lab8();
        break;
case 34:
        ex4Lab8();
        break;
case 35:
        filtru_median();
        break;
case 36:
        filtru_gaussian();
        break;
case 37:
        gradFX();
        break;
case 38:
        gradFY();
        break;
case 39:
        modulSiDirectie();
        break;
case 40:
        canny_method();
        break;
default:
        break;
    }
} while (op != 0);
return 0;
}

```